

Министерство науки и высшего образования РФ
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

ОТЧЁТ ПО КУРСОВОМУ ПРОЕКТУ

по дисциплине «Технологии разработки качественного программного обеспечения»

Модульное тестирование веб-сайта Kitchen Planner

Выполнил

студент гр. 5130202/20202 Селиванов С.Ю.

Руководитель Маслаков А.П.

Санкт-Петербург

2026 г.

1. Описание выполненной работы

1.1. Общая характеристика проекта

Веб-приложение «Автоматизированный планировщик кухонного гарнитура» предназначено для формирования компоновки кухни на основании параметров помещения, выбранных пользователем модулей и набора конструктивных ограничений. Пользователь задаёт размеры кухни, тип планировки, бытовую технику, параметры верхних шкафов и дополнительные пожелания текстом. Система нормализует входные данные, фиксирует явно выбранные параметры, применяет правила автоматической оптимизации и возвращает набор модулей вместе с SVG-чертежами.

Архитектура проекта включает клиентскую часть на React/Next.js, API-маршруты Next.js, серверное приложение FastAPI, генератор прямой кухни, генератор угловой кухни, модуль оптимизации длин, подсистему формирования верхних и нижних модулей, обработчик текстовых пожеланий и SVG-рендерер. Отдельно реализована поддержка авторизации, личного кабинета и истории генераций пользователя.

Цель модульного тестирования на текущем этапе — проверить наиболее рискованные участки логики: разбор пользовательских пожеланий, группировку верхних шкафов, работу угловой генерации, формирование SVG-представлений и корректность проксирующих API-маршрутов frontend.

Компонент	Технологии	Назначение
Frontend	React, Next.js, TypeScript	Интерфейс ввода параметров кухни, фиксация выбранных значений, вывод предупреждений, истории и SVG-эскизов.
API routes	Next.js route handlers	Проксирование запросов от интерфейса к backend-сервису генерации и распознавания пожеланий.
Backend	Python, FastAPI	HTTP API, генерация планировки, обработка истории, авторизация и интеграция с базой данных.
Генератор	Python	Построение прямой и угловой кухни, подбор модулей, оптимизация размеров и применение ограничений.
Рендерер	Python, SVG	Построение top view, front view и side view с размерами, фасадами, техникой и условными обозначениями.
База данных	PostgreSQL, SQLAlchemy	Хранение пользователей и истории генераций.
LLM-подсистема	Ollama + правила	Распознавание текстовых пожеланий пользователя и

		преобразование их в фиксируемые параметры формы.
--	--	--

1.2. Используемые инструменты

Инструмент	Назначение
pytest	Запуск модульных тестов backend-логики.
pytest-cov	Формирование отчёта покрытия backend-кода.
Vitest	Запуск модульных тестов frontend API-маршрутов.
@vitest/coverage-v8	Формирование отчёта покрытия frontend-тестов.
jsdom	Тестовая DOM-среда для frontend-тестирования.
React Testing Library	Библиотека для будущего тестирования React-компонентов пользовательского интерфейса.
Next.js build	Проверка компиляции и сборки frontend-приложения.
python compileall	Проверка синтаксической корректности Python-модулей backend.
HTML coverage report	Визуальная проверка покрытия тестами по файлам и строкам.

1.3. Применённые техники тест-дизайна

1.3.1. Классы эквивалентности

Входные данные были разделены на классы эквивалентности по типам пользовательских параметров и ожидаемому поведению генератора. Такой подход позволяет проверять не каждую комбинацию параметров, а представителей классов, которые должны обрабатываться одинаково.

- тип кухни: прямая и угловая;
- тип верхних шкафов: распашные и подъёмные;
- размещение СВЧ: соло, встроенная в колонну, встроенная в верхний шкаф;
- верхние модули: обычный upper cabinet, сушка, шкаф вытяжки, антресоль;
- короткий верхний модуль: объединяемый с соседом и не объединяемый с соседом;
- текстовые пожелания: явно распознаваемые правилами, неявные фразы, некорректные значения;
- ответ backend: успешная генерация и ошибка engine.

1.3.2. Граничные значения

Для модулей кухни важны граничные значения ширин: минимальные размеры drawer-модулей, минимальная ширина складного подъёмника 500 мм, ширины варочной

поверхности 300/600/800/900 мм, ширины посудомоечной машины 450/600 мм. В тестах отдельно проверяется случай короткого верхнего сегмента 250 мм, который должен объединяться с соседним модулем либо переводиться в распашной фасад, если объединение невозможно.

1.3.3. Негативное тестирование

Негативные проверки направлены на ситуации, в которых система должна не падать, а корректно вернуть ошибку или безопасный fallback. Например, API-маршрут `/api/plan` должен обернуть ошибку backend engine в понятный JSON-ответ, а подсистема распознавания пожеланий должна игнорировать неизвестные поля и недопустимые значения.

1.3.4. Изоляция зависимостей

Frontend-тесты изолируют внешний backend с помощью mock-реализации fetch. Это позволяет проверять поведение API routes независимо от фактического запуска FastAPI-сервиса. Backend-тесты проверяют чистые функции генерации, рендера и нормализации параметров без обращения к базе данных и без запуска HTTP-сервера.

2. Отчёт о прохождении тестов и покрытии

2.1. Backend (Python / FastAPI)

Для backend-модуля создан набор unit-тестов в каталоге `engine/tests`. Проверяются правила группировки верхних модулей, fallback-поведение коротких подъёмников, работа генератора угловой кухни с верхней встроенной СВЧ, корректность формирования SVG-представлений и нормализация результатов распознавания пожеланий.

```
cd C:\Users\Степан\kitchen-planner\engine
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m pytest --cov=app --cov-report=term --cov-report=html
```

Тестовый файл	Проверяемая область	Ключевые проверки
test_wall_modules.py	Формирование верхних шкафов	Объединение коротких lift-сегментов, объединение с сушкой, fallback в распашной фасад.
test_generator_renderer.py	Генератор и рендерер	Единая высота верхних шкафов при СВЧ в верхнем модуле, построение всех SVG-видов угловой кухни.
test_llm_preferences.py	Распознавание пожеланий	Извлечение JSON, фильтрация недопустимых patch-значений, определение подъёмных верхних фасадов из естественной фразы.

По результатам запуска backend-набора выполнено 8 тестов, все тесты пройдены успешно. Общее покрытие backend-кода составило 63%. Покрытие распределено неравномерно: модули генерации и рендера покрыты лучше, чем авторизация и работа с базой данных, так как текущий отчёт посвящён в первую очередь логике планировщика.

```
PS C:\Users\Сренан\kitchen-planner\engine> .\venv\Scripts\python.exe -m pytest [100%]
===== warnings summary =====
.venv\Lib\site-packages\_pytest\cacheprovider.py:475
C:\Users\Сренан\kitchen-planner\engine\.venv\Lib\site-packages\_pytest\cacheprovider.py:475: PytestCacheWarning: could
not create cache path C:\Users\Сренан\kitchen-planner\engine\.pytest_cache\v\cache\nodeids: [WinError 183] Невозможно с
оздать файл, так как он уже существует: 'C:\Users\Сренан\kitchen-planner\engine\.pytest_cache\v\cache'
  config.cache.set("cache/nodeids", sorted(self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
8 passed, 1 warning in 4.00s
```

Рисунок 1. Успешный запуск backend unit-тестов

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform win32, python 3.14.3-final-0
-----
Name                               Stmts  Miss  Cover
-----
app\__init__.py                     0      0   100%
app\auth.py                         130    130     0%
app\corner_generator.py             457     60    87%
app\database.py                     19     19     0%
app\generator.py                    334     88    74%
app\kitchen_constants.py            26      0   100%
app\kitchen_options.py              173     62    64%
app\length_optimizer.py             258    192    26%
app\llm_preferences.py              191    126    34%
app\lower_modules.py                707    381    46%
app\main.py                          27     27     0%
app\models.py                       7      7     0%
app\prompt_parser.py                 37     37     0%
app\renderer.py                     1198    258    78%
app\wall_modules.py                 426     92    78%
-----
TOTAL                               3990   1479    63%
Coverage HTML written to dir htmlcov
8 passed, 1 warning in 5.77s
```

Рисунок 2. Покрытие backend-кода в терминале



Рисунок 3. HTML-отчёт покрытия backend-кода

2.2. Frontend (Next.js / TypeScript)

Для frontend-части создан набор unit-тестов API-маршрутов Next.js. На данном этапе проверяются маршруты /api/plan и /api/preferences/parse, так как они являются связующим слоем между пользовательским интерфейсом и backend engine. Внешний backend в тестах заменяется mock-реализацией fetch.

```
cd C:\Users\Степан\kitchen-planner\web
npm test
npm run test:coverage
```

Тестовый файл	Проверяемая область	Ключевые проверки
api/plan/route.test.ts	API-маршрут генерации	Проксирование успешного ответа engine и корректная обработка неуспешного HTTP-ответа.
api/preferences/parse/route.test.ts	API-маршрут распознавания пожеланий	Передача текста пожеланий в backend parser и возврат patch-результата клиенту.

По результатам запуска frontend-набора выполнено 3 теста, все тесты пройдены успешно. Покрытие протестированных API routes составило 87.5% по statements и lines, 100% по functions. Дальнейшее расширение набора должно включать тесты React-компонентов формы и личного кабинета.

```
PS C:\Users\Степан\kitchen-planner\web> npm test

> web@0.1.0 test
> vitest run

RUN v4.1.7 C:/Users/Степан/kitchen-planner/web

✓ src/app/api/preferences/parse/route.test.ts (1 test) 9ms
✓ src/app/api/plan/route.test.ts (2 tests) 10ms

Test Files  2 passed (2)
Tests       3 passed (3)
Start at    04:28:31
Duration    1.28s (transform 78ms, setup 242ms, import 144ms, tests 19ms, environment 1.81s)
```

Рисунок 4. Успешный запуск frontend unit-тестов

```
PS C:\Users\Степан\kitchen-planner\web> npm run test:coverage

> web@0.1.0 test:coverage
> vitest run --coverage

RUN v4.1.7 C:/Users/Степан/kitchen-planner/web
Coverage enabled with v8

✓ src/app/api/preferences/parse/route.test.ts (1 test) 11ms
✓ src/app/api/plan/route.test.ts (2 tests) 12ms

Test Files  2 passed (2)
Tests       3 passed (3)
Start at    04:29:29
Duration    1.28s (transform 73ms, setup 247ms, import 134ms, tests 23ms, environment 1.67s)

% Coverage report from v8
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	87.5	58.33	100	87.5	
plan	90	62.5	100	90	
route.ts	90	62.5	100	90	31
preferences/parse	83.33	50	100	83.33	
route.ts	83.33	50	100	83.33	17

Рисунок 5. Покрытие frontend-кода в терминале

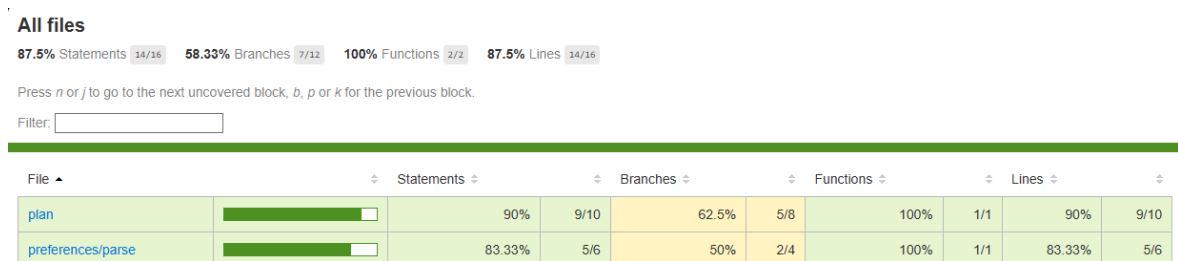


Рисунок 6. HTML-отчёт покрытия frontend-кода

2.3. Итоговая сводка покрытия

Подсистема	Команда	Количество тестов	Покрытие	Комментарий
Backend	pytest --cov=app	8	63%	Покрыты ключевые правила генератора, wall_modules, renderer и LLM preferences.
Frontend	vitest run --coverage	3	87.5%	Покрыты API routes /api/plan и /api/preferences/pars e.
Сборка frontend	npm run build	-	успешно	Next.js-приложение компилируется после добавления тестовой инфраструктуры.
Компиляция backend	python -m compileall engine/app	-	успешно	Python-модули проходят синтаксическую проверку.

3. Процедура расширения тестового набора

3.1. Пример: расширение backend-тестов для нового правила верхних шкафов

При добавлении нового правила генерации рекомендуется сначала выделить чистую функцию или локальный сценарий, который можно проверить без запуска сервера. Например, для правила объединения коротких складных подъёмников был добавлен тест, передающий в apply_wall_facade_grouping короткий upper cabinet шириной 250 мм и соседний модуль шириной 600 мм.

```
def test_lift_upper_merges_short_segment_with_regular_neighbor():
    modules = [
        _wall_module('upper_cabinet', 400, 250),
        _wall_module('upper_cabinet', 650, 600),
    ]
    result = apply_wall_facade_grouping(modules, upper_cabinet_opening='lift')
    assert result[0]['width_mm'] == 850
    assert result[0]['facade_opening'] == 'lift_fold'
```

Такой тест фиксирует ожидаемое поведение на уровне доменного правила и защищает его от регрессий при дальнейшей переработке угловой генерации или SVG-рендера.

3.2. Рекомендации по дальнейшему развитию тестов

- Добавить unit-тесты для `length_optimizer`: уменьшение мойки, варочной поверхности, посудомоечной машины и изменение вариаций духовки/СВЧ.
- Добавить тесты для `lower_modules`: `cutlery drawer`, `filler drawer`, `drawer_1/drawer_2/drawer_3` и запрет нестандартного изменения базовых модулей.
- Добавить тесты для `corner_generator`: перенос модулей между сторонами, запрет соседства `dishwasher/oven/upper microwave` с `corner base`, выбор `corner tall`.
- Добавить тесты `renderer`: отсутствие подписей у скрытых модулей, корректные размерные стрелки, отсутствие наложений и корректная отрисовка фальш-планок.
- Добавить frontend component-тесты: изменение параметров формы, автоматическая фиксация выбранных пользователем модулей, история генераций, загрузка и удаление генерации.
- Добавить тесты авторизации и личного кабинета с mock-базой или тестовой PostgreSQL-базой.
- Добавить end-to-end тесты Playwright для сценариев: регистрация, генерация кухни, сохранение результата, открытие истории и скачивание PNG.

Заключение

В результате выполненной работы для проекта подготовлена начальная инфраструктура модульного тестирования backend и frontend-частей. Тестовый набор покрывает наиболее критичные правила текущей реализации: группировку верхних модулей, поведение складных подъёмников, генерацию угловой кухни, SVG-рендеринг и API-прокси Next.js. Полученные HTML-отчёты покрытия могут использоваться для дальнейшего планирования расширения тестового набора.

Текущее покрытие backend составляет 63%, frontend API-слоя — 87.5%. Эти показатели достаточны для первичного отчёта и демонстрации применённых техник тестирования, однако дальнейшая работа должна быть направлена на покрытие авторизации, базы данных, UI-компонентов и end-to-end сценариев взаимодействия пользователя с системой.